Check out the expect line. It reads almost like its English equivalent: "We expect the sandwich's taste to be delicious." With a traditional test framework's assertions, you might write a line like the following one instead:

01-getting-started/02/sandwich_test.rb
```
assert_equal('delicious', taste, 'Sandwich is not delicious')
```

This code works fine, but in our opinion it is less clear than the RSpec version. Throughout this book we are going to be harping on keeping your specs readable.

Specs are also working code. You should be able to run them and check that the sandwich is really behaving as designed. In the next section, you'll do so.

Understanding Failure

To try your specs, run the rspec command from your project directory. RSpec will look inside the spec subdirectory for files named «something»_spec.rb and run them:

```
$ rspec
F

Failures:

  1) An ideal sandwich is delicious
     Failure/Error: sandwich = Sandwich.new('delicious', [])

     NameError:
       uninitialized constant Sandwich
     # ./spec/sandwich_spec.rb:4:in `block (2 levels) in <top (required)>'

Finished in 0.00076 seconds (files took 0.08517 seconds to load)
1 example, 1 failure

Failed examples:

rspec ./spec/sandwich_spec.rb:3 # An ideal sandwich is delicious
```

RSpec gives us a detailed report showing which spec failed, the line of code where the error occurred, and a description of the problem.

Additionally, the output is in color. RSpec uses color to emphasize different parts of the output:

- Passing specs are green.
- Failing specs, and failure details, are red.
- Example descriptions and structural text are black.
- Extra details such as stack traces are blue.
- Pending specs (which we'll see later in *Marking Work in Progress*, on page 26) are yellow.